

HYUNDAI MOTOR · 차량사이버보안개발팀

Day 2

MCP · 보안 도구 자동화

통합 운영자료 (개념+실습 병행) · 6/9 火 · 09:00–17:00

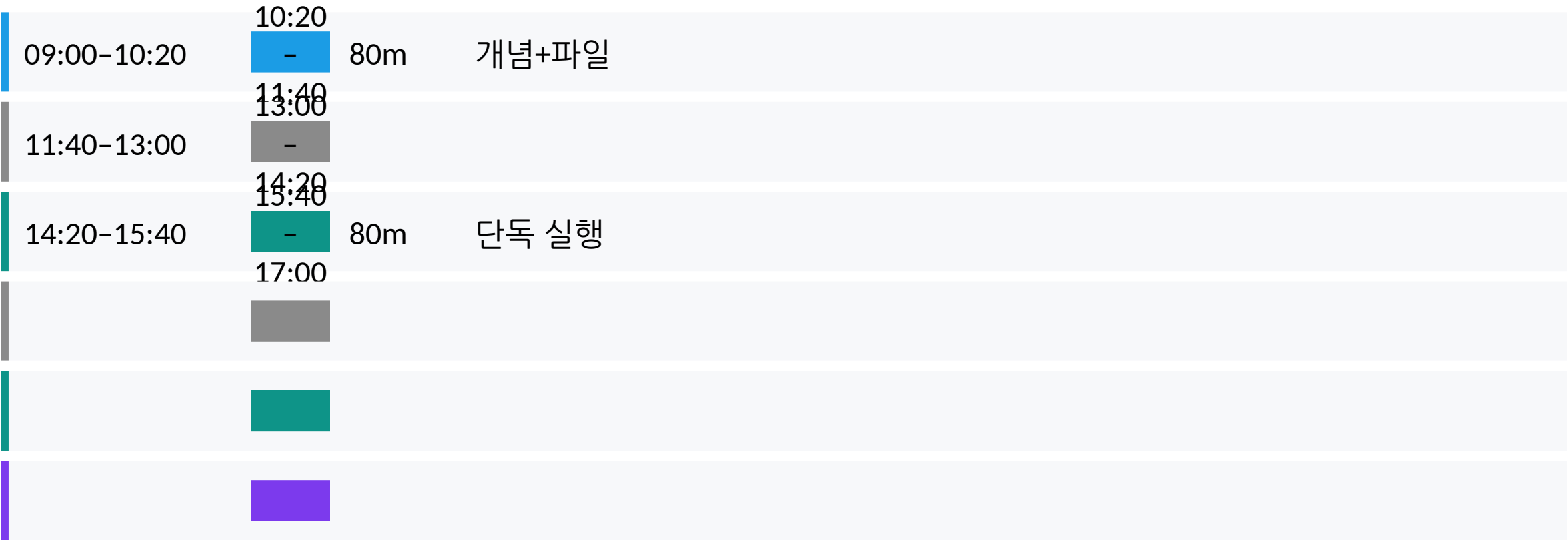
보안 도구를 MCP 표준으로 연결하고, 도구의 발견과 LLM의 해석을 결합해 리포트를 자동화한다.

이 파일 하나로 하루 수업을 끝까지 진행합니다. 상세 이혼은 *Appendix*를 참조하세요.

모두의연구소 · Agentic Security 심화 과정 · 2주차 (Day 1~5)

Day 2 운영 흐름

보안 도구를 MCP 표준으로 연결하고, 도구의 발견과 LLM의 해석을 결합해 리포트를 자동화한다.



Day 2 — 무엇을 배우고 무엇을 만드는가

학습 목표·실습 목표·사용 파일·성공 기준을 시작 전에 꼭 봐야 합니다.

학습 목표 (개념·실습 병행)

- 에이전트 루프와 컨텍스트 관리
- MCP 등장 배경과 도구 호출 표준
- 도구의 발전 vs LLM의 해석 차이

실습 목표 (실습·산출물 확인)

- 필수: run_lab2_mcp_demo.py로 도구 3종 호출
- 확장: security-tools MCP 등록·대화창 호출
- 도구 출력(JSON)을 해석 리포트(md)로 변환

사용 파일

- scripts/run_lab2_mcp_demo.py (필수)
- mcp_servers/security_tools_server.py
- agents/secret_agent.py
- agents/dependency_agent.py

성공 기준

- 필수: lab2_scan_raw.json·lab2_report.md 생성
- 확장: claude mcp list에 security-tools 표시
- 도구 원본(JSON)과 해석(md) 분리 저장
- MCP 등록 실패 시 fallback으로 진행 가능

5일 로드맵: 단일 Agent에서 통합 Security PoC까지

오늘이 전체 과정에서 어디에 위치하는지 먼저 확인합니다.

단일 Agent에서 통합 Security PoC까지 이어지는 하나의 파이프라인입니다.



① 개념 설명 → ② 파일 확인 → ③ 직접 실행 → ④ 산출물 확인 → ⑤ 관찰 정리 → ⑥ 다음 연결

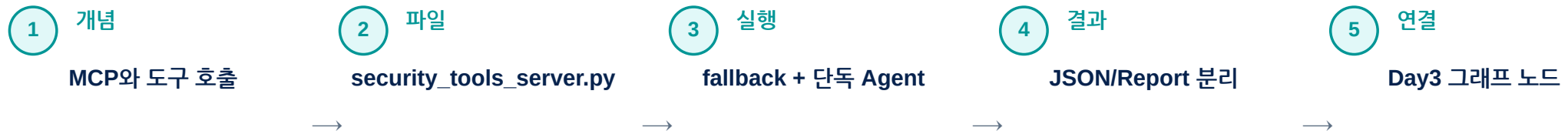
개념 + 실습 병행 운영 패턴 (09:00-17:00)

개념을 설명하고 바로 파일 확인·실행·산출물 검증으로 이어갑니다.

수업은 이론을 몰아서 듣는 방식이 아니라, 작은 학습 루프를 하루에 3회 이상 반복합니다.



매일 최소 3회 이상 직접 실행



운영 메모: Hooks 관점: 도구 실행 전후에 로그·검증·마스킹을 자동으로 붙이는 구조를 익힌다.

오늘의 자료 구조: PPT → 코드 → 명령 → 산출물

수강생이 지금 보는 장표가 어떤 파일과 어떤 결과물로 이어지는지 명확히 보여줍니다.

PPT의 개념은 실제 Codespaces 파일·명령어·산출물로 바로 연결되어야 합니다.



PPT 개념

도구를 붙인 Agent

MCP · Tool Calling · Raw
JSON

확장 개념
Tool Design, Context Budget,
Hooks



관련 파일

mcp_servers/
security_tools_server.py

scripts/run_lab2_mcp_demo.py

agents/secret_agent.py

agents/dependency_agent.py



실행 명령

```
python scripts/run_lab2_mcp_demo.py --target  
sample_app/
```

```
python agents/secret_agent.py --target  
sample_app/
```

```
python agents/dependency_agent.py --target  
sample_app/
```



산출물

reports/lab2_scan_raw.json

reports/lab2_report.md

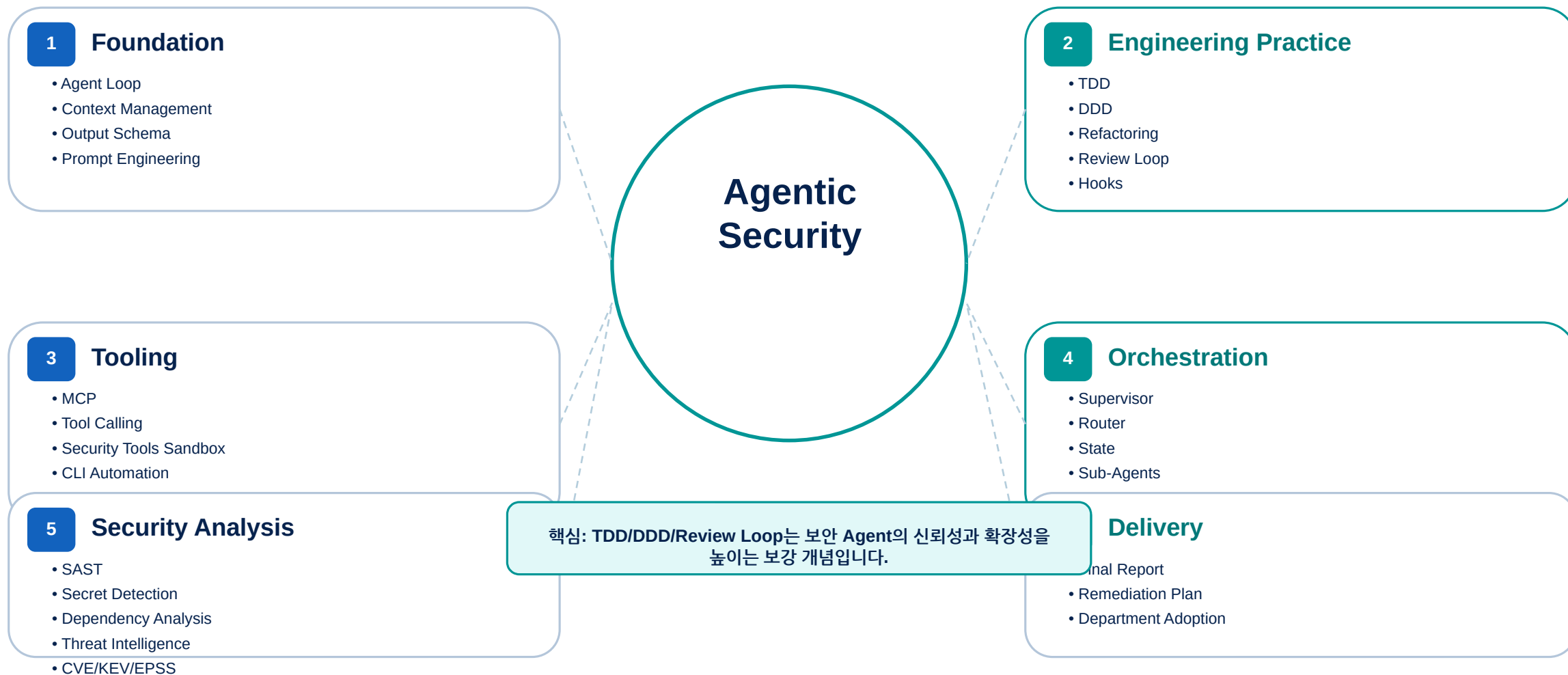
reports/lab2_secret_report.md

reports/
lab2_dependency_report.md

보강 개념 맵: 실전 에이전틱 보안 엔지니어링

TDD·DDD·Review Loop·Hooks 등은 현재 실습 흐름을 보강하는 개념으로 연결합니다.

레퍼런스의 실전 에이전틱 코딩 패턴을 현재 보안 실습 흐름에 맞게 재구성합니다.



이 그림은 오늘의 핵심 구조를 파일·실행·산출물 관점으로 연결합니다.



Mini Lab → Main Lab 연결

작은 실험으로 개념을 먼저 체감하고, 바로 보안 실습으로 확장합니다.

Mini Lab · Toy Tool Calling

count_words
extract_keywords
Raw JSON
tool_calling_result.md

Main Lab · MCP security-tools

MCP Server
scan_code
detect_secrets
lab2_report.md

연결 문장

Toy Tool에서 확인한 Raw JSON/Report 분리는 security-tools의 lab2_scan_raw.json/lab2_report.md 구조로 확장됩니다.

Mini Lab 실행 · Toy Tool Calling

한 줄 명령으로 실행하고, 산출물을 확인합니다.

왜 하는가

MCP 전에 도구 호출과 원본/해석 분리를
아주 작은 예제로 확인합니다.

```
python course/mini_labs/day02_tool_calling/run_toy_tool_demo.py
```

관련 파일

```
course/mini_labs/day02_tool_calling/  
run_toy_tool_demo.py  
toy_tools.py  
sample_feedback.txt
```

생성 산출물

```
reports/tool_calling_raw.json  
reports/tool_calling_result.md
```

확인 포인트

도구 원본 JSON 확인
Markdown 해석 리포트 확인
MCP 본실습과 비교

Tool Calling · Context Budget · Hooks

부가 개념을 키워드가 아니라 실습·산출물·검증과 연결합니다.

Tool Calling

LLM이 모든 일을 직접 하지 않습니다.
결정적 계산·추출은 도구가 맡고, Agent는 결과를 해석합니다.

Context Budget

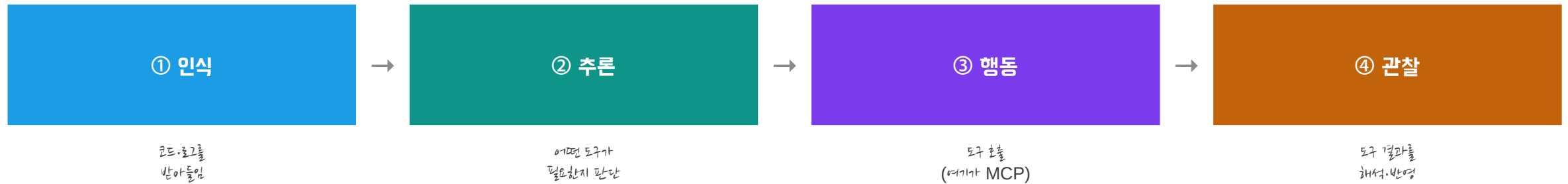
Claude/GPT에는 전체 레포가 아니라 필요한 산출물·context pack만 넘깁니다.
저토큰 운영의 핵심입니다.

Hooks 관점

도구 실행 전후에 입력 범위 확인, 결과 저장, 마스킹, audit log 기록을 붙입니다.

에이전트 루프 도구는 '행동' 단계에서 쓰인다

여기서 본 인식→추론→행동→관찰의 순환에서, 도구 호출이 정확히 어디에 들어가는지 짚습니다.



도구 호출은 ③ 행동 단계에서 일어납니다. MCP는 이 '행동'을 표준화하는 약속입니다.

컨텍스트 관리 무엇을 도구에 넘기고 무엇을 가릴까

도구를 부를 때도 하네스 요소 ② (컨텍스트 관리)이 작동합니다. 보안에서는 특히 중요합니다.

①

필요한 것만 전달

점점 대상 코드 경로만 도구에 넘깁니다. 무관한 파일을 함께 넘기면 비용·노이즈가 늘고 분석이 느려집니다.

②

민감 정보 차단

도구로 넘기는 입력에서도 비밀 값을 마스킹합니다. 어제 SAST 리포트 마스킹과 같은 원칙입니다.

③

출력 분리 보존

도구의 원본 출력(JSON)과 LLM 해석(md)을 분리해 둡니다. 해석이 플러그 원본으로 되돌아갈 수 있어야 합니다.

Claude Code에서 컨텍스트를 아끼는 세 가지 습관

도구를 다루기 전에, 에이전틱 코딩 도구를 쓸 때 컨텍스트를 낭비하지 않는 실전 습관을 익힙니다.

①

새 작업은 새 대화로

이전 작업의 긴 이력이 남아 있으면 판단이 흐려지고 비용이 늘어납니다. 주제가 바뀌면 대화를 새로 시작합니다.

②

긴 결과는 파일로

수백 줄짜리 스캔 결과를 대화에 계속 들고 다니지 말고, 파일(reports/)로 저장한 뒤 필요한 부분만 다시 읽게 합니다.

③

계획 먼저, 실행은 나중

복잡한 작업은 먼저 계획을 세우게 하고 사람이 확인한 뒤 실행시킵니다. 계획 단계가 곧 작은 HITL입니다.

이 세 습관은 오늘 실습·산출물 확인에서 그대로 쓰입니다. 특히 ②는 '원본 JSON 분리 보존' 원칙과 같은 뿌리입니다.

MCP — 왜 '도구 호출 표준'이 필요한가

도구마다 호출 방식이 제각각이면, 도구 하나 추가에 큰 작업이 듭니다. MCP는 이 혼란을 표준으로 정리합니다.

MCP 이전

- 도구마다 다른 호출 방식·형식
- 도구 추가에 매번 연결 코드 작성
- 에이전트와 도구가 강하게 묶임

MCP 이후

- 표준 프로토콜로 호출 방식 통일
- 도구 추가 = 표준 서버에 등록 한 번
- 에이전트와 도구가 느슨하게 분리

MCP(Model Context Protocol)는 'LLM이 외부 도구를 부르는 방식'의 공통 약속입니다. USB 표준이 주변기기를 통일한 것과 같습니다.

MCP(도구 호출) vs Orchestration(조율)

둘은 자주 혼동되지만 역할이 다릅니다. 이 구분이 오늘과 내일(Day 3)을 가릅니다.

MCP

도구를 '어떻게 부르나'

개별 도구 하나를 표준 방식으로 호출하는 게 중요. 오늘 다룹니다. scan_code를 부르는 법이 여기에 해당합니다.

Orchestration

도구·Agent를 '어떤 순서로'

여러 Agent와 도구를 어떤 순서로, 누구 책임 하에 조율할지. 내일(Day 3) Supervisor가 맡습니다.

핵심

오늘의 위치

오늘은 '도구를 표준으로 연결'하는 데까지입니다. 그 도구들을 지휘하는 것은 내일의 몫입니다.

어제 우리 MCP 서버는 '순수 보안 도구'만 호출하기로 설계했습니다. 에이전트를 부르는 리모컨이 아니라, 도구 그 자체입니다.

왜 외부 도구 대신 Security Tools Sandbox인가

본 과정은 외부 보안 도구를 직접 끌어오는 대신, 통제 가능한 자체 도구 묶음(MCP 서버)을 사용합니다. 그 이유를 분명히 합니다.

①

통제 가능성

교육 환경에서는 도구의 동작을 완전히 예측·재현할 수 있어야 합니다. 자체 샌드박스(security-tools)는 입력·출력·권한이 모두 우리 손 안에 있습니다.

②

읽기 전용 안전성

외부 도구는 권한 범위가 넓을 수 있습니다. 우리 도구 4종은 분석만 하고 코드를 바꾸지 않는 읽기 전용이라, 실습 사고를 원천 차단합니다.

③

확장 경로

사내에서 쓰는 외부 도구가 필요하면, 같은 MCP 표준으로 서버 하나를 더 등록하면 됩니다. 구조를 바꾸지 않고 도구만 늘립니다.

정리: 본 과정은 'Security Tools Sandbox'(security-tools MCP 서버)를 표준 도구로 씁니다. 외부 도구 연동이 필요하면 동일한 MCP 방식으로 추가하면 됩니다.

실습 루프 (LAB 2)

LAB 2 — security-tools MCP 서버 연결

이제 파일로만 존재하던 MCP 서버를 실제로 등록하고, 도구를 호출해 리포트를 만듭니다.

오늘의 실습 계단 LAB 2-1 → 2-2 → 2-3 (+확장)

도구화된 단일 Agent들을 하나씩 둘러봅니다. 내일(Day 3) Supervisor가 뒤를 부품을 오늘 전부 만져봅니다.

2-1

도구 3종 데모

필수 실습
run_lab2_mcp_demo
원본/해석 분리

2-2

Secret 단독

secret_agent.py
비밀 탐지·마스킹
SAST와의 차이

2-3

Dependency 단독

dependency_agent.py
라이브러리 CVE
후보 추론

확장

MCP 등록

claude mcp add
대화형 호출
(CLI 준비된 사람)

오늘 단독으로 돌린 SAST(어제)·Secret·Dependency가 내일 Supervisor 그래프의 노드가 됩니다. Day 3은 새 개념이 아니라 '오늘까지의 부품을 조립하는 날'입니다.

LAB 2가 하는 일 한눈에

오늘 실습의 목표·준비물·사용 파일·성공 신호를 먼저 정리합니다.

목표

- 보안 도구를 MCP로 등록
- 도구 발견 + LLM 해석 결합

준비물

- Day 1 환경(check_env 통과)
- Claude Code/Codex 로그인

사용 파일

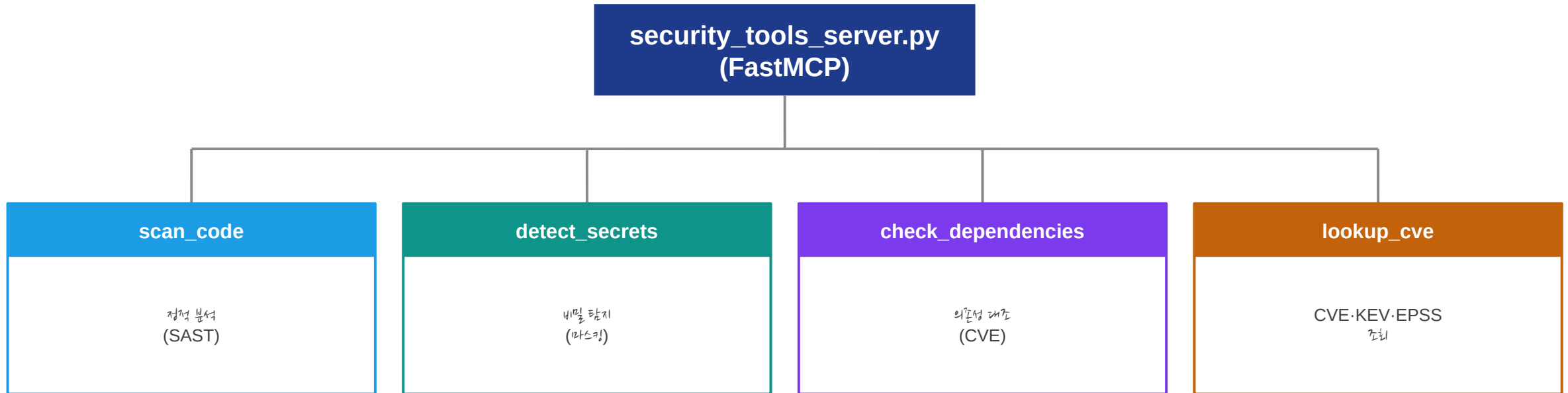
- mcp_servers/security_tools_server.py
- agents/ 도구 4종

성공 신호

- claude mcp list에 표시
- 원본 JSON·해석 md 분리

security_tools_server.py는 어떻게 생겼나

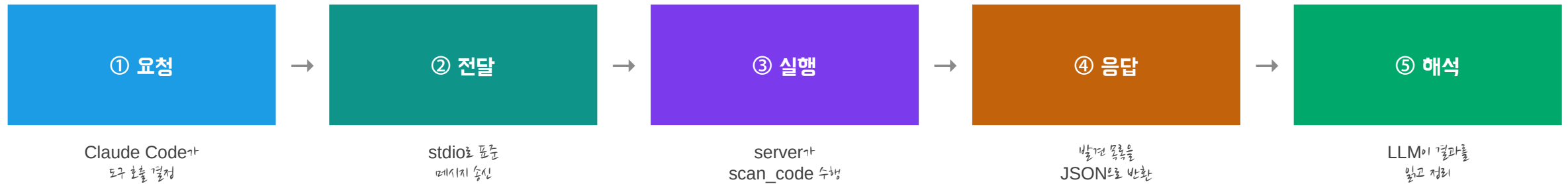
MCP 서버 한 파일이 네 개의 순수 보안 도구를 노출합니다. 에이전트가 아니라 '도구'라는 점이 핵심입니다.



각 도구는 `@mcp.tool()` 데코레이터로 등록됩니다. 어제 만든 Agent의 분석 함수를 도구로 감싼 것입니다.

그림으로 보는 MCP 통신 한 번의 도구 호출

claude mcp add 뒤에 실제로 무슨 일이 오가는지, 한 번의 scan_code 호출을 따라갑니다.



핵심: ①~④는 표준 약속(MCP)이라 도구가 무엇이든 같은 모양입니다. 도구를 바꿔도 ⑤의 해석 방식은 그대로 이것이 표준화의 이득입니다.

좋은 도구의 세 가지 조건

도구를 직접 만들거나 고를 때의 기준입니다. 우리 security-tools 4종이 이 기준을 따릅니다.

①

이름이 곧 설명

scan_code, detect_secrets처럼
이름만 봐도 하는 일이 보이게.
에이전트가 올바른 도구를
고르는 첫 단서입니다.

②

좁은 입출력

입력은 경로 하나, 출력은
정해진 JSON 형태.
넓은 만큼 도구보다 좁은
전문 도구가 오류가 적습니다.

③

최소 권한

점점 도구는 읽기 전용.
수정·배치 권한은 주지 않음.
도구의 권한이 곧
에이전트의 권한입니다.

체크 질문: '이 도구가 오작동하면 최악의 피해는 무엇인가?' — 답이 무서울수록 권한을 더 좁혀야 합니다.

필수 실습 스크립트로 도구 3종 직접 호출

Claude CLI 인증·MCP 등록 앱에서도 Day 2 핵심 학습을 진행합니다. ZIP의 scripts/와 1:1 일치합니다.

실행

```
python scripts/run_lab2_mcp_demo.py --target sample_app/
```

출력 · 산출물

```
[tool] scan_code           → 9건  
[tool] detect_secrets     → 3건  
[tool] check_dependencies → 4건  
→ reports/lab2_scan_raw.json (도구 원본)  
→ reports/lab2_report.md     (해석 리포트)  
→ memory/audit_log.jsonl     (호출 기록 3건)
```

이 스크립트는 MCP 서버에 등록된 것과 동일한 도구 함수를 직접 호출합니다. CLI가 막혀도 '도구의 발견 + 원본/해석 분리'라는 오늘의 핵심은 그대로 배웁니다. 이어지는 확장 실습에서 같은 도구를 Claude Code로 호출합니다.

LAB 2-2 — Secret Agent 단독 실행

비밀 탐지 전문 Agent를 단독으로 돌립니다. SAST와 무엇이 다른지가 관찰의 핵심입니다. ZIP 명령과 1:1 일치.

실행

```
python agents/secret_agent.py --target sample_app/
```

출력 (값은 항상 마스킹)

```
[Secret] 노출 의심 3건 (값은 마스킹됨)
[확정   ] vulnerability.py:31 하드코딩 자격증명 API_TOKEN = "sk-****"
[의심   ] vulnerability.py:66 비밀 추정 변수 logging.info("log****", ...)
→ reports/lab2_secret_report.md 저장
```

관찰 포인트: SAST는 '위험한 코드 패턴'을, Secret Agent는 '비밀의 존재'를 봅니다. 같은 파일에서 다른 것을 찾는 두 전문가 이 역할 차이가 Day 3 분업의 근거입니다.

LAB 2-3 — Dependency Agent 단독 실행

우리 코드가 아니라 '우리가 쓰는 라이브러리'의 위험을 봅니다. Day 4 위협 정보의 입력이 여기서 나옵니다.

실행

```
python agents/dependency_agent.py --target sample_app/
```

출력

```
[Dependency] 취약 의존성 4건  
[High      ] CVE-2020-14343 pyyaml 5.1 취약 버전  
[Medium    ] CVE-2024-22195 jinja2 2.11.2 취약 버전  
→ reports/lab2_dependency_report.md 저장
```

관찰 포인트: 여기서 추출된 CVE 목록이 Day 4 threat_agent의 입력이 됩니다. '코드 점검'과 '공급망 점검'은 다른 표면을 보는 다른 전문가입니다.

필수와 확장 두 갈래 실습 구조

어떤 환경에서도 수업이 멈추지 않도록 실습을 두 갈래로 운영합니다.

필수 실습 (전원)

- **scripts/run_lab2_mcp_demo.py 실행**
- CLI 인증 불필요 — 항상 동작
- 도구 원본·해설·감사 기록 생성

확장 실습 (CLI 준비된 사람)

- `claude mcp add security-tools -- python mcp_servers/security_tools_server.py`
- `claude mcp list`로 등록 확인
- 대화창에서 **scan_code** 자연어 호출

운영 원칙: 필수 실습으로 전원이 산출물을 확보한 뒤, 확장 실습은 준비된 수강생부터 진행합니다. CLI 문제가 생겨도 수업은 계속됩니다.

확장 실습 MCP 서버 등록 (claude mcp add)

필수 실습을 마친 뒤 진행하는 확장 실습입니다. Claude CLI 로그인이 전제되며, 명령어는 ZIP 실제 경로와 1:1 일치합니다.

등록 명령

```
claude mcp add security-tools -- python mcp_servers/security_tools_server.py
```

등록 확인

```
claude mcp list
security-tools  ✓ connected
  tools: scan_code, detect_secrets, check_dependencies, lookup_cve
```

주의: mcp라는 폴더명은 사용하지 않습니다. 반드시 mcp_servers 폴더명을 사용합니다 외부 mcp 패키지와의 이름 충돌을 피하기 위한 설계입니다.

확장 실습 Claude Code 대화창에서 도구 호출

등록된 도구는 자연어로 부를 수 있습니다. 에이전트가 적절한 도구를 골라 호출합니다.

대화창 입력

```
> sample_app/vulnerability.py를 scan_code로 점검하고  
심각도 순으로 정리해줘
```

도구 호출 확인 (Claude Code가 보여주는 흐름)

```
● Calling tool: scan_code(target="sample_app/vulnerability.py")  
● Tool returned 8 findings  
→ LLM이 심각도 순 정리·해석 추가
```

도구 호출이 실제로 일어났는지는 'Calling tool' 표시로 확인합니다. 이것이 안 보이면 도구가 아니라 LLM이 지어낸 답일 수 있습니다.

세 가지 산출물 원본·해석·기록

도구를 부르면 세 가지가 남아야 합니다. 이 분리가 검증과 추적의 기반입니다.

①

도구 원본

scan_raw.json

스캐너 출력 그대로.
수정하지 않는 '증거'.

②

해석 리포트

lab2_report.md

LLM이 심각도 순 정리.
맥락·권고를 더한 문서.

③

감사 기록

audit_log.jsonl

어떤 도구를 어떤 인자로
불렀는지 한 줄 기록.

원본과 해석을 분리하는 이유: 해석이 틀렸을 때 원본 JSON으로 돌아가 검증할 수 있어야 하기 때문입니다(Provenance의 시작).

도구 깊이 보기 scan_code · detect_secrets

등록된 도구 네 개 중 오늘 핵심으로 쓰는 둘을 자세히 봅니다.

scan
_code

정적 분석 (SAST)

소스 코드를 규칙으로 훑어 취약 패턴을 찾습니다. 입력은 점진적 경로, 출력은 발견 목록(위치·CWE·심각도·근거). 어제 만든 sast_agent.run을 도구로 감싼 것입니다.

detect_secrets

비밀 탐지

하드코딩된 키·토큰·비밀번호를 찾습니다. 출력 시 값은 반드시 마스킹합니다(sk-****). 어제 배운 마스킹 원칙이 도구 차원에서도 지켜집니다.

도구 깊이 보기 check_dependencies · lookup_cve

나머지 두 도구는 외부 위험 정보와 연결되는 다리입니다. Day 4에서 본격적으로 쓰입니다.

chec
k_de
pend
enci
es
look
up_c
ve

의존성 취약점 대조

requirements.txt의 패키지 버전을 취약점 DB와 대조해 영향받는 패키지와 CVE를 산출합니다. 코드가 아니라 '쓰는 라이브러리'의 위험을 봅니다.

CVE·KEV·EPSS 조회

CVE 식별자로 KEV 등재 여부와 EPSS 점수를 조회합니다. 오늘은 도구 존재만 확인하고, Day 4에서 위험도 점수화에 본격 활용합니다.

네 도구가 어제 만든 다섯 Agent의 분석 능력을 'MCP 표준'으로 외부에 노출한 것입니다. 도구와 에이전트는 같은 로직의 두 얼굴입니다.

도구 원본 JSON은 어떻게 생겼나

해석 리포트만 보지 말고 원본 JSON의 구조를 이해해야, 해석이 틀렸을 때 되짚을 수 있습니다.

reports/lab2_scan_raw.json (발취)

```
[
  {
    "rule": "SQLI-STRCAT", "severity": "High",
    "cwe": "CWE-89", "file": "vulnerability.py", "line": 39,
    "evidence": "query = ...", "confidence": "의심",
    "provenance": "tool:sast_rules#SQLI-STRCAT"
  }, ...
]
```

provenance 필드에 주목하세요. '이 발견이 어느 도구·규칙에서 나왔는가'를 기록합니다. 이것이 Day 3의 Provenance 추적으로 이어집니다.

Plan Mode — 실행 전에 계획을 먼저 본다

에이전틱 코딩 도구를 안전하게 쓰는 핵심 습관입니다. 도구를 바로 실행시키지 않고 계획을 먼저 검토합니다.

①

계획 단계 분리

복잡한 작업은 '무엇을 어떤 순서로 할지' 계획을 먼저 세우게 하고, 사람이 그 계획을 확인한 뒤 실행에 들어갑니다.

②

작은 HITL

계획 검토 자체가 일종의 사전 승인입니다. Day 3에서 배울 HITL의 가벼운 버전이 여기서 미리 등장합니다.

③

비용 절감

잘못된 방향으로 도구를 여러 번 호출하기 전에 계획에서 바로잡으면, 토큰·시간 낭비를 크게 줄입니다.

원칙: 되돌리기 어려운 작업일수록 '계획 먼저, 실행 나중'. 보안 점검은 신중함이 속도보다 우선입니다.

도구의 발견과 LLM의 해석은 다르다

둘을 구분하는 것이 우리의 핵심 통찰입니다. 각자 잘하는 일이 다릅니다.

도구가 잘하는 것 (결정적)

- 같은 입력 → 항상 같은 발견
- 패턴·변전 등 사실 확인
- 재현 가능·검증 가능

LLM이 잘하는 것 (해석적)

- 발견들을 맥락으로 묶어 설명
- 우선순위·권고 제시
- 사람이 읽을 문장으로 정리

결합으로만 가능한 것: '결정적 발견에 근거한 신뢰할 수 있는 해석'. 도구 없는 LLM은 지어내고, LLM 없는 도구는 나열만 합니다.

MCP 등록이 안 될 때 복구 절차

강의장에서 가장 흔히 막히는 지점입니다. 증상별로 차근차근 점검합니다.

①

목록에 안 보임

claude mcp list에 security-tools가 없으면, claude mcp add 명령을 경로(mcp_servers/)까지 정확히 확인해 재실행합니다.

②

등록됐는데 호출 실패

python mcp_servers/security_tools_server.py를 단독 실행해 import 오류가 없는지 봅니다. 도구 코드를 수정하다 문법 오류를 낸 경우가 많습니다.

③

도구가 fallback 출력

외부 mcp 패키지가 설치됐는지 확인합니다(pip install mcp). 폴더명이 mcp_servers/인지도 확인합니다.

④

키 관련 오류

.env 또는 Secrets에 키가 있는지 확인합니다. 도구 자체는 키 없이도 동작하지만, LLM 해석에는 키가 필요합니다.

LAB 2 회고 우리 업무로 가져가기

오늘 배운 'MCP로 도구 표준화'를 부서 업무와 연결하는 5분 회고입니다.

관찰한 것

- 도구는 사실임, LLM은 의미를 담당
- 원본과 해석을 나눠 보존해야 검증 가능
- MCP 표준으로 도구 추가가 쉬워짐

가져갈 질문

- 우리 팀이 이미 쓰는 보안 도구를 MCP로 묶는다면?
- 도구 출력 중 마스킹이 필요한 민감 정보는?
- 어떤 도구를 '읽기 전용'으로 제한해야 하나?

여기서 적은 답이 Day 5 '부서 적용 구상'의 재료가 됩니다.

Day 2 — 오늘 만든 산출물 확인

오늘 저장소에 실제로 남은 것들입니다. 다음 Day의 입력적으로 누적됩니다.

01

등록된 MCP 서버

claude mcp add security-tools -- python mcp_servers/security_tools_server.py 로 등록된 보안 도구 4종

02

reports/ 도구 산출물

도구 원본 JSON과 LLM 해석 md를 분리 보존 — 검증·역추적의 출발점

MCP 서버가 연결되면, Day 3의 오케스트레이터가 이 도구들을 그대로 호출하게 됩니다.

Day 2 — 자주 나는 오류

강의장에서 가장 흔히 막히는 지점과 즉시 대응법입니다.

claude CLI 없음/로그인 안 됨

→ 필수 실행(run_lab2_mcp_demo.py)으로 진행, 확장은 설치 후

claude mcp list에 도구 안 보임

→ claude mcp add 재실행, 경로(mcp_servers/) 확인

import 오류로 서버 기동 실패

→ python mcp_servers/security_tools_server.py 단독 실행해 문법 오류 확인

도구는 호출되나 결과 빈약

→ --target 경로·sample_app 존재 확인

원본 JSON과 해석 md가 한 파일에 섞임

→ 원본/해석 분리 저장 원칙 준수

Day 2 — 완료 체크리스트 · 다음 Day 연결

체크리스트로 마감하고, 내일로 이어지는 연결고리를 확인합니다.

- ✓ 필수: `run_lab2_mcp_demo.py` 실행 성공
- ✓ `lab2_scan_raw.json`·`lab2_report.md` 생성 확인
- ✓ 확장: `claude mcp list`에 `security-tools` 표시
- ✓ 확장: 대화창에서 `scan_code` 호출 확인
- ✓ 원본과 해석 분리 보존 이유 설명
- ✓ MCP 등록 실패 시 fallback 절차 숙지

다음 Day 연결고리 → Day 3: 도구를 진 여러 Agent를 누가 어떤 순서로 지휘하는가 오케스트레이션

APPENDIX

부록 이론 상세 보충

본문에서 압축한 개념의 상세 설명입니다. 수업 중 질문이 나오거나, 복습 시 참조하세요.

[상세] 도구 활용 5유형

본문 '이론 1'의 상세판. 에이전트가 쓰는 도구를 다섯 유형으로 나눠 봅니다.

유형	설명	보안 점검 예
조회/검색	정보를 읽어옴	lookup_cve, check_kev
분석/스캔	대상을 검사	scan_code, detect_secrets
계산/변환	데이터 가공	위험도 점수 계산
생성/작성	산출물 만들기	리포트 md 생성
실행/조작	환경 변경	(본 과정 미부여 — 읽기 전용 원칙)

본 과정의 보안 도구는 '실행/조작'을 제외한 읽기 전용입니다. 점검 에이전트에 코드 수정·배포 권한을 주지 않는 최소 권한 원칙입니다.

[상세] MCP의 동작 클라이언트와 서버

본문 '이론 3'의 상세판. MCP가 실제로 어떻게 통신하는지 한 겹 더 들어갑니다.

①

MCP 서버

도구를 호출하는 쪽. 우리 security_tools_server.py가 여기에 해당합니다. stdio로 떠서 요청을 기다립니다.

②

MCP 클라이언트

도구를 부르는 쪽. Claude Code/Codex가 클라이언트입니다. 등록된 서버의 도구 목록을 받아 호출합니다.

③

표준 메시지

들 사이의 대화는 정해진 형식(JSON 기반)을 따릅니다. 그래서 어떤 도구든 같은 방식으로 부를 수 있습니다.

[상세] 왜 도구와 LLM을 결합하는가

본문 'LAB 2 발원 vs 해석'의 이론적 배경입니다.

도구만 한계

패턴은 잡지만 '왜 위험한지', '무엇부터 고칠지'를 설명하지 못합니다. 나열에 그칩니다.

LLM 만 위험

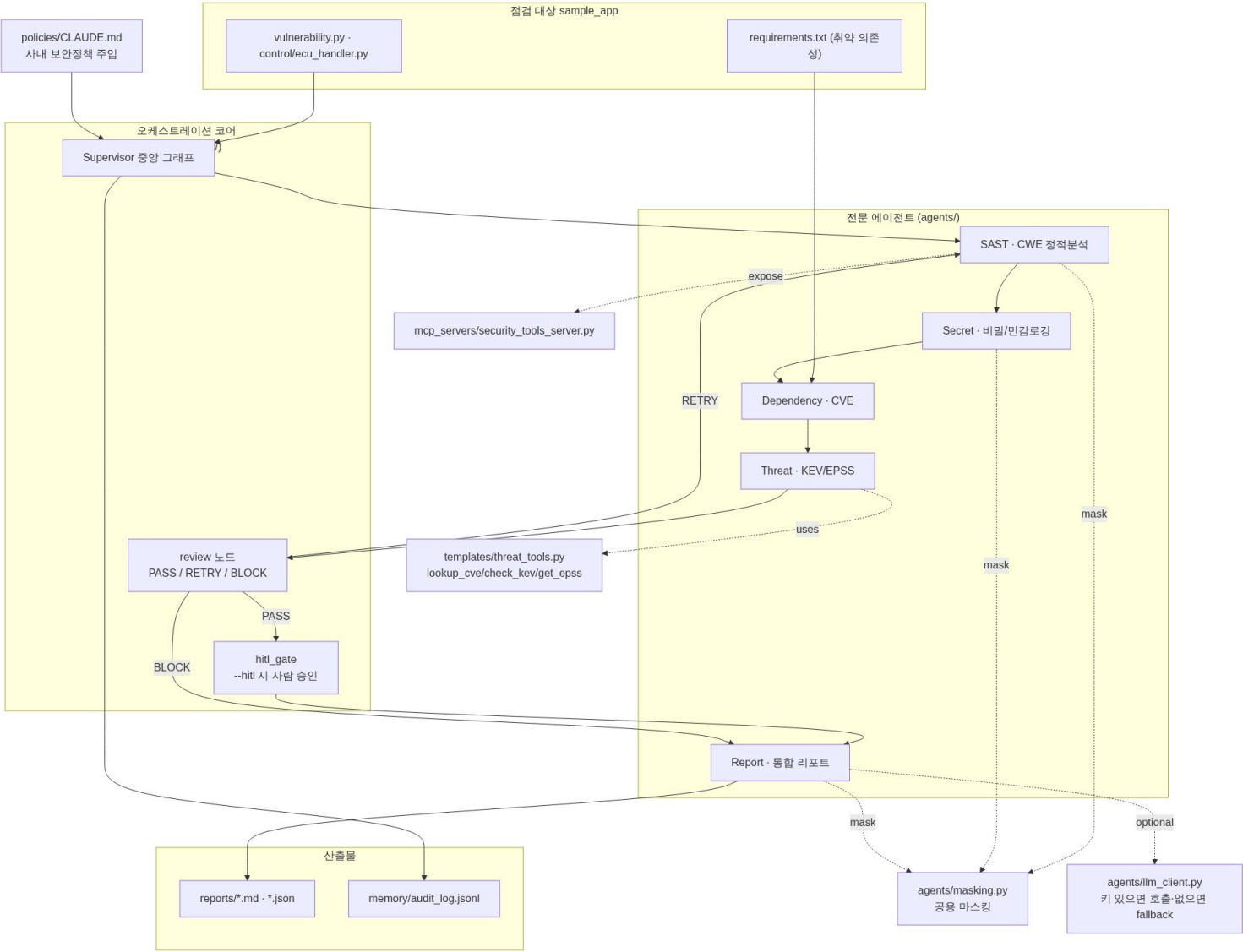
그럴듯한 설명을 만들지만, 근거 없이 지어낼(hallucination) 수 있습니다. 보안에서는 치명적입니다.

결합 해답

도구의 결정적 발원을 LLM이 해석합니다. '근거 있는 설명'이 되어, 신뢰와 가독성을 모두 얻습니다.

이것이 본 과정이 '도구를 가진 에이전트'를 강조하는 이유입니다. 도구는 사실을, LLM은 의미를 담당합니다.

전체 아키텍처 — Security Agent Lab



Day 2 동작 흐름 (시퀀스 다이어그램)

